

Guía Recomendada: Solución + Aplicación de Consola en .NET 10

Objetivo

Aprender a crear una solución en .NET 10 con una aplicación de consola, comprender su estructura, ejecutar proyectos, organizar código en múltiples capas y aplicar buenas prácticas reales de desarrollo.

Introducción conceptual

Antes de ejecutar comandos, el alumno debe entender:

¿Qué es una solución (`.sln`)?

Una solución permite agrupar varios proyectos relacionados:

- Aplicación principal
- Bibliotecas de clases
- Tests unitarios
- Otros módulos

¿Qué es un proyecto?

Un proyecto contiene código fuente compilable:

- Consola
- Web API

- Biblioteca
- Test

2 Estructura profesional recomendada

ConsoleSolution/

|

|— src/

| └─ ConsoleApp/

|

|— tests/

| └─ ConsoleApp.Tests/

|

└─ ConsoleSolution.sln

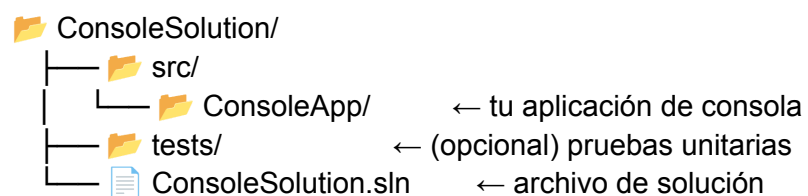
Explicación:

Carpeta	Propósito
src	Código productivo
tests	Pruebas automatizadas

.sln	Archivo solución
------	------------------

Escenario: Solución + Aplicación de Consola (.NET 8)

Paso 1: Estructura de carpetas



Paso 2: Comandos (ejecuta en terminal)

```
# 1. Crear carpeta raíz
mkdir ConsoleSolution
cd ConsoleSolution

# 2. Crear la solución
dotnet new sln --name ConsoleSolution

# 3. Crear la aplicación de consola
dotnet new console --name ConsoleApp --output src/ConsoleApp --framework net10.0

# 4. Agregar el proyecto a la solución
dotnet sln add src/ConsoleApp/ConsoleApp.csproj

# 5. Verificar estructura
tree /F # Windows
# o
ls -R # Linux/macOS
```



Paso 3: Compilar y ejecutar

```
# Compilar toda la solución
dotnet build

# Ejecutar la aplicación
dotnet run --project src/ConsoleApp/ConsoleApp.csproj

# O desde la carpeta del proyecto
cd src/ConsoleApp
dotnet run
```

Salida esperada:

```
Hello, World!
```



Paso 4: Personalizar la aplicación de consola

Edita `src/ConsoleApp/Program.cs` con un ejemplo más interesante:

```
// Versión moderna (top-level statements)
Console.WriteLine("=== Mi Aplicación de Consola ===\n");

Console.Write("Ingresa tu nombre: ");
string? nombre = Console.ReadLine();

Console.Write("Ingresa tu edad: ");
string? edadInput = Console.ReadLine();

if (int.TryParse(edadInput, out int edad))
{
    Console.WriteLine($"{nombre}! Tienes {edad} años.");
    Console.WriteLine($"El año que viene tendrás {edad + 1} años.");
}
else
{
    Console.WriteLine($"{nombre}! (No ingresaste una edad válida)");
}
```

```
Console.WriteLine("\nPresiona cualquier tecla para salir...");
Console.ReadKey();
```

Paso 5: Configuración avanzada

Agregar argumentos de línea de comandos

Modifica `Program.cs`:

```
// Verificar argumentos pasados al ejecutar
string[] args = Environment.GetCommandLineArgs();

if (args.Length > 1)
{
    Console.WriteLine($"Argumentos recibidos: {string.Join(", ", args[1..])}");
}
else
{
    Console.WriteLine("No se recibieron argumentos");
}

// Ejemplo: dotnet run -- --nombre Juan --edad 25
```

Agregar colores

```
Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Texto en verde");
Console.ResetColor();
```

Paso 6: Agregar múltiples proyectos a la solución

```
# Agregar un proyecto de biblioteca de clases (desde la carpeta de la solución)
cd ../..
dotnet new classlib --name ConsoleLibrary --output src/ConsoleLibrary --framework
```

```
net10.0
dotnet sln add src/ConsoleLibrary/ConsoleLibrary.csproj

# Agregar referencia de la biblioteca a la consola
dotnet add src/ConsoleApp/ConsoleApp.csproj reference
src/ConsoleLibrary/ConsoleLibrary.csproj
```

Usar la biblioteca en tu consola:

src/ConsoleLibrary/Utils.cs:

```
namespace ConsoleLibrary;

public static class Utils
{
    public static string GetGreeting(string name) => $"¡Hola, {name}!";
}
```

src/ConsoleApp/Program.cs:

```
using ConsoleLibrary;

Console.WriteLine(Utils.GetGreeting("Mundo"));
```



Paso 7: Publicar ejecutable standalone

```
# Publicar para Windows (exe independiente)
dotnet publish src/ConsoleApp/ConsoleApp.csproj -c Release -r win-x64 --self-contained
true -p:PublishSingleFile=true -o ./publish

# Para Linux
dotnet publish src/ConsoleApp/ConsoleApp.csproj -c Release -r linux-x64 --self-contained
true -p:PublishSingleFile=true -o ./publish

# Para macOS
dotnet publish src/ConsoleApp/ConsoleApp.csproj -c Release -r osx-x64 --self-contained
true -p:PublishSingleFile=true -o ./publish

# El ejecutable estará en la carpeta ./publish
```

--



Paso 8: Agregar pruebas unitarias

```
# Crear proyecto de pruebas
dotnet new xunit --name ConsoleApp.Tests --output tests/ConsoleApp.Tests --framework
net10.0
dotnet sln add tests/ConsoleApp.Tests/ConsoleApp.Tests.csproj

# Agregar referencia al proyecto principal
dotnet add tests/ConsoleApp.Tests/ConsoleApp.Tests.csproj reference
src/ConsoleApp/ConsoleApp.csproj

# Ejecutar pruebas
dotnet test
```


Resumen de los comandos



Comandos útiles resumidos

```
# Crear solución y consola (FLUJO COMPLETO)
mkdir ConsoleSolution && cd ConsoleSolution
dotnet new sln --name ConsoleSolution
dotnet new console --name ConsoleApp --output src/ConsoleApp
--framework net10.0
dotnet sln add src/ConsoleApp/ConsoleApp.csproj

# Compilar
dotnet build

# Ejecutar
dotnet run --project src/ConsoleApp/ConsoleApp.csproj

# Ejecutar con argumentos
dotnet run --project src/ConsoleApp/ConsoleApp.csproj -- --arg1 valor1
--arg2 valor2

# Limpiar
dotnet clean

# Ver proyectos en la solución
dotnet sln list
```

Primeras clases:

Crear una aplicación de consola

```
dotnet new console --name ConsoleApp --framework net10.0
```

Crear una solución que contenga una aplicación de consola:

```
# Crear la carpeta de trabajo
mkdir ConsoleSolution cd ConsoleSolution
# Crear la solución dotnet new sln --name ConsoleSolution
# Crear el proyecto de consola
dotnet new console --name ConsoleApp --output src/ConsoleApp --framework net10.0
# Agregar el proyecto a la solución
dotnet sln add src/ConsoleApp/ConsoleApp.csproj
```

Crear una solución con una aplicación de consola y un proyecto biblioteca de clases.

```
# Crear la carpeta de trabajo
mkdir ConsoleLibrarySolution
cd ConsoleLibrarySolution
# Crear la solución dotnet new sln --name ConsoleLibrarySolution
# Crear la aplicación de consola
dotnet new console --name ConsoleApp --output src/ConsoleApp --framework net10.0
# Crear la biblioteca de clases
dotnet new classlib --name MisClases --output src/MisClases --framework net10.0
# Agregar ambos proyectos a la solución
dotnet sln add src/ConsoleApp/ConsoleApp.csproj
dotnet sln add src/MisClases/MisClases.csproj
# Agregar la referencia desde la consola hacia la biblioteca dotnet add
src/ConsoleApp/ConsoleApp.csproj reference src/MisClases/MisClases.csproj
```